THE THREE-COLOR RAMSEY NUMBER OF THE 10-VERTEX PATH

P. Graham
https://orcid.org/0009-0007-8493-1505

Abstract. For a graph H, the three-color Ramsey number $R(H,H,H)$ is the least N such that every coloring of the edges of the complete graph K_N with three colors contains a monochromatic copy of H. Faudree and Schelp (J. Combin. Theory Ser. B 19, 1975) conjectured that $R(P_n,P_n,P_n) = 2n-2+(n \bmod 2)$ for the path P_n on n vertices. Gyárfás, Ruszinkó, Sárközy and Szemerédi proved this for all sufficiently large n, and Dybizbański, Dzido and Radziszowski determined $R(P_8,P_8,P_8) = 14$ and $R(P_9,P_9,P_9) = 17$, completing the cases $n \le 9$ and leaving $n = 10$ as the first open case. We prove $R(P_{10},P_{10},P_{10}) = 18$: every 3-coloring of the edges of K_{18} contains a monochromatic path on 10 vertices. The proof is a SAT computation: symmetry breaking anchored on a monochromatic P_9, whose existence is re-derived by the same method for the smaller cases, path constraints generated during the search, and a cube-and-conquer split into 15,956 subproblems, each with an unsatisfiability certificate. The argument is formalized in Lean 4; every certificate is checked by Lean's verified LRAT checker, run by native evaluation.

AI production. The proofs, the Lean formalization, the search code, the certificate checkers and the text were produced by AI models (from OpenAI and Anthropic), working under the author's direction.
Scope of verification. This is a computer-assisted proof. The statement, the reduction from colorings to propositional formulas, the cover of the case split, and the lower bound are proved in Lean from the definitions. The unsatisfiability certificates are checked inside Lean by native evaluation, which adds Lean's compiler to the trusted base (Section 6). The corresponding refutations, before their conversion to the compact form Lean reads, were also checked independently of Lean by the verified checker `cake_lpr` (Section 7). No human expert has reviewed the proof.

1. Introduction

1.1. **The problem.** For graphs G_1, G_2, G_3, the Ramsey number $R(G_1,G_2,G_3)$ is the least N such that every coloring of the edges of K_N with colors $1,2,3$ contains, for some i, a copy of G_i all of whose edges have color i. We write $R_3(G) = R(G,G,G)$, and P_n for the path on n vertices (with $n-1$ edges). Paths are not required to be induced.

Faudree and Schelp [13] studied $R(P_m,P_n,P_k)$ for paths of different lengths and suggested that the diagonal case obeys a simple formula for all n; Dybizbański, Dzido and Radziszowski [7] stated it as a conjecture, and Radziszowski's dynamic survey [26] records it as the conjecture of Faudree and Schelp.

Conjecture 1.1 (Faudree–Schelp [13]; see [7, 26])**.** *For all $n \ge 1$, $R_3(P_n) = 2n-2+(n \bmod 2)$.*

The lower bound $R_3(P_n) \ge 2n-2+(n \bmod 2)$ holds for every n, by Faudree and Schelp's construction [13], a blow-up of the factorization of K_4 into three perfect matchings (see also [7, 15] and Section 2). Figaj and Łuczak [12] determined $R(P_{n_1},P_{n_2},P_{n_3})$ asymptotically, and Gyárfás, Ruszinkó, Sárközy and Szemerédi [15] proved Conjecture 1.1 for all sufficiently large n, by the regularity method, with no explicit threshold; Dybizbański, Dzido and Radziszowski report that their attempts to extract a reasonable bound on the threshold from that proof failed [7, Section 2]. Letzter [21] later gave a simpler proof for long paths, also for large n.

Date: October 21, 2026.

For small n the conjecture is known for $n \le 9$. Table 1 lists the values with their sources as the survey [26, Section 6.4.1(c)] attributes them. The values for P_5, P_6 and P_7 were obtained with the help of computer algorithms [7, Section 2]. Dybizbański, Dzido and Radziszowski [7] (preprint 2012, published 2016) proved $R_3(P_8) = 14$ and $R_3(P_9) = 17$ without computer help, which, in their words, "leaves $R_3(P_{10})$ as the first open case". They also tried to treat P_{10} and P_{11} by the same method, found the case analysis much harder, and expected the even case $R_3(P_{10}) = 18$ to be the more difficult of the two [7, Section 5]. The latest revision of Radziszowski's survey (revision #18, April 24, 2026) still states that the conjecture "is true for $n \le 9 \ldots$, and the first open case is that for P_{10}" [26, Section 6.4].

n	3	4	5	6	7	8	9
$R_3(P_n)$	5	6	9	10	13	14	17
source	[26, Ea1]	[19]	[33]	[33]	[32]	[7]	[7]

TABLE 1. The known values of $R_3(P_n)$, with the sources the survey [26] gives. For $n = 3$ the survey notes that the value is easy to obtain.

For comparison, counting edges with Faudree and Schelp's extremal theorem for paths [13] (see also [7, Theorem 4]) gives an upper bound: a P_{10}-free graph on 23 vertices has at most $2\binom{9}{2}+\binom{5}{2} = 82$ edges, and $3 \cdot 82 < \binom{23}{2}$, so $R_3(P_{10}) \le 23$.

1.2. **The result.**

Theorem 1.2. *$R_3(P_{10}) = 18$. That is, every coloring of the edges of K_{18} with three colors contains a monochromatic path on 10 vertices, and some 3-coloring of the edges of K_{17} contains none.*

The lower bound is the classical construction; the new part is the upper bound $R_3(P_{10}) \le 18$, which settles the case $n = 10$ of Conjecture 1.1. The result is likely new: we found it neither in the survey, which lists the case as the first open one [26, Section 6.4], nor in the literature we searched. The companion case $n = 11$, $R_3(P_{11}) = 21$, remains open; Dybizbański, Dzido and Radziszowski showed that it would follow from the conjectured value $R_3(C_{10}) = 20$ for the 10-cycle [7], which is itself the first open case of Dzido's conjecture $R_3(C_{2m}) = 4m$ for even cycles [9], [26, Section 6.3.1(b)]; Benevides and Skokan [3] proved that conjecture for large m.

The proof is a computation, built directly on the work of Dybizbański, Dzido and Radziszowski. A monochromatic P_9 exists in every 3-coloring of K_{18} because $R_3(P_9) = 17$ [7], and fixing it is the first and most effective reduction of the search. We do not take that bound from the literature: a ladder of seven smaller computations, P_3 on K_5 up to P_9 on K_{17}, re-derives the upper bounds $R_3(P_m) \le 5, 6, 9, 10, 13, 14, 17$ for $m = 3, \ldots, 9$, each rung fixing the monochromatic path that the rung below guarantees (Section 4). The top rung, P_{10} on K_{18}, is split into 15,956 subproblems ("cubes"); each is refuted by a SAT solver with a proof certificate, and a separate certificate shows that the cubes cover all cases (Section 5).

Lean 4 [24] with Mathlib [23] proves the reduction from colorings to formulas: from the unsatisfiability of the rungs, the cover formula and the cubes, it derives that every 3-coloring of K_{18} has a monochromatic P_{10}. Every certificate (all 15,956 cubes, the cover and the seven rungs) is checked inside Lean by the verified LRAT checker that ships with Lean (it underlies the `bv_decide` tactic [1]), run by native evaluation. The final theorem is the Lean statement `ramsey3Path 10 = 18` (Section 6).

1.3. **Related and concurrent work.**

SAT solvers with proof certificates have settled several Ramsey-type questions, among them the Boolean Pythagorean triples problem [16], by cube-and-conquer [17], and the three-color Ramsey number $R(4, 3, 3) = 30$ [4]; the empty hexagon theorem [18] was later verified in Lean [27]. Szeider has recently described a tool that imports LRAT

certificates, including those of cube-and-conquer computations, into Lean theorems through the same verified checker of Lean's core library [28]; our certificate import (Section 6.3) relies on the same checker, with its own compact certificate encoding. On multicolor cycle Ramsey numbers, Wesley [30] used SAT solvers to prove $R(C_3, C_6, C_6) = R(C_5, C_6, C_6) = 15$ together with new bounds, and Galindez [14] has posted an exhaustive SAT computation, using SAT Modulo Symmetries [20] and cube-and-conquer, giving $R_3(C_{10}) \le 26$. That bound implies $R_3(P_{10}) \le 26$, since a 10-cycle contains a P_{10}. A public SAT project on three-color cycle Ramsey numbers [34] is also active. Three-color Ramsey numbers of paths of different lengths, and of paths against cycles, have been determined in many further cases [9, 8, 25, 22], and Radziszowski's survey [26] records many further computer-assisted results on small Ramsey numbers.

1.4. **Organization.** Section 2 gives the lower bound. Section 3 describes the propositional encoding and the two symmetry reductions, Section 4 the ladder, and Section 5 the case split on K_{18} and why it is complete. Section 6 describes the Lean formalization and exactly what it trusts; Section 7 the checks made independently of Lean, and which steps of that route are machine-checked and which are argued. Section 8 gives the computing cost and how to replay the proof.

2. The lower bound

Partition $\{0, \dots, 16\}$ into $A = \{0, \dots, 4\}$, $B = \{5, \dots, 8\}$, $C = \{9, \dots, 12\}$ and $D = \{13, \dots, 16\}$. Color an edge with color 0 if both ends lie in $A \cup B$ or both lie in $C \cup D$; with color 1 if it joins A to C or B to D; and with color 2 if it joins A to D or B to C. This is the blow-up of the factorization of K_4 into the matchings $\{AB, CD\}$, $\{AC, BD\}$, $\{AD, BC\}$, with the edges inside the parts given color 0: Faudree and Schelp's construction [13] for $n = 10$ (see also [7, 15]).

Proposition 2.1. *This coloring of K_{17} has no monochromatic P_{10}. Hence $R_3(P_{10}) \ge 18$.*

Proof. For each color, the vertex set splits into two classes with no edge of that color between them: $A \cup B$ and $C \cup D$ for color 0, $A \cup C$ and $B \cup D$ for color 1, $A \cup D$ and $B \cup C$ for color 2. Each class has 8 or 9 vertices, so a monochromatic path, which stays inside one class, has at most 9 vertices. □

In Lean the coloring is written out edge by edge, and the argument is a general lemma (for all n and m: if each color has a labeling of the vertices that is constant along edges of that color and whose classes have fewer than m vertices, there is no monochromatic P_m) together with two finite facts decided by the kernel's own reduction (`decide +kernel`), without native evaluation.

3. The encoding

Throughout, a *coloring* of K_n is a map X from the edges to $\{0, 1, 2\}$, and it is *good* (for m) if it has no monochromatic P_m. We order the edges $\{i, j\}$, $i < j$, row by row: $\{0,1\}, \{0,2\}, \dots, \{0, n-1\}, \{1,2\}, \dots, \{n-2, n-1\}$, and number them $e = 0, 1, \dots, \binom{n}{2} - 1$ in this order.

3.1. **Variables and the coloring clauses.** The variable $x_{e,c} = 3e + c + 1$ means "edge e has color c". For each edge there is one clause $x_{e,0} \vee x_{e,1} \vee x_{e,2}$ and three clauses $\neg x_{e,c} \vee \neg x_{e,d}$, $c < d$, so that models correspond to colorings. For a color c and distinct vertices $p_0, \dots, p_{m-1}$, the *path clause*

$$\bigvee_{i=0}^{m-2} \neg x_{\{p_i, p_{i+1}\}, c}$$

says that this path is not monochromatic in color c. A coloring is good exactly when its assignment satisfies every path clause. The path clauses are not written into the formula in advance; they are generated during the search (Section 3.4).

3.2. **The fixed path.** Vertex permutations $\pi \in S_n$ and color permutations $\sigma \in S_3$ act on colorings by $(g \cdot X)(\{\pi a, \pi b\}) = \sigma(X(\{a, b\}))$ for $g = (\pi, \sigma)$. They map good colorings to good colorings.

Lemma 3.1 (fixed path)**.** *Let $k \le n$ and suppose every 3-coloring of K_n contains a monochromatic P_k. Then every good coloring of K_n is equivalent under $S_n \times S_3$ to a good coloring X with $X(\{t, t+1\}) = 0$ for $0 \le t \le k-2$.*

Proof. Take a monochromatic path $v_0, \dots, v_{k-1}$ in color c; apply a vertex permutation sending v_t to t and the transposition of colors c and 0. □

The formula for the rung "P_m on K_n with a fixed P_k" contains the $k-1$ unit clauses $x_{\{t,t+1\},0}$. Only the path edges are constrained; the chords of the fixed path are free.

3.3. **Lex-leader constraints.** Compare colorings by the lexicographic order $\le_{\text{lex}}$ of their color vectors $(X(e))_e$ in edge order, with $0 < 1 < 2$. Let Γ be a list of generators $g = (\pi, \sigma)$. With a fixed P_k on $0, \dots, k-1$ and $W = \{k, \dots, n-1\}$, Γ consists of the transpositions of pairs of vertices of W (in lexicographic order), the reversal $t \mapsto k-1-t$ of $\{0, \dots, k-1\}$, and the color transposition $1 \leftrightarrow 2$. On the first rung, K_5, nothing is fixed and Γ consists of all vertex transpositions and the color transpositions $0 \leftrightarrow 1$ and $1 \leftrightarrow 2$.

Lemma 3.2 (lex-leader)**.** *Let S be the set of good colorings with color 0 on the fixed path. Each $g \in \Gamma$ maps S to itself. Consequently, if S is nonempty it contains a coloring X^* with $X^* \le_{\text{lex}} g \cdot X^*$ for every $g \in \Gamma$.*

Proof. When a path is fixed, each vertex permutation in Γ maps the path's edge set to itself and each color permutation in Γ fixes color 0, so Γ preserves the fixed path; every g preserves goodness. (With nothing fixed, S is the set of all good colorings.) Take X^* to be the $\le_{\text{lex}}$-least coloring reachable from a member of S by finitely many moves from Γ; it lies in S, and $g \cdot X^*$ is reachable too, so $X^* \le_{\text{lex}} g \cdot X^*$. □

The point of Lemma 3.2 is that the lex-leader constraints for *every* generator hold simultaneously for one coloring that also has the fixed path; the two reductions are therefore jointly sound. The generators do not need to generate the full stabilizer of the fixed path: a weaker set of constraints only leaves more symmetry in the search.

The constraint $X \le_{\text{lex}} g \cdot X$ is encoded in the standard way [5]. The positions compared are the edges f in edge order, except those with $\pi^{-1}(f) = f$ and σ the identity (where $X(f) = (g \cdot X)(f)$ identically). For the j-th compared position f_j, let $Y_j = \sigma(X(\pi^{-1} f_j))$ be the color of $g \cdot X$ there, and let q_j be an auxiliary variable meaning "X and $g \cdot X$ agree on the first j compared positions" (q_0 is true and omitted). The clauses are, for colors a, b:

$$q_{j-1} \to \neg(X(f_j) = a \wedge Y_j = b) \quad (a > b), \qquad q_{j-1} \wedge X(f_j) = a \wedge Y_j = a \to q_j,$$
$$q_j \to q_{j-1}, \qquad q_j \wedge X(f_j) = a \to Y_j = a.$$

Under the intended meaning of the q_j, every clause holds exactly when $X \le_{\text{lex}} g \cdot X$. For K_{18} with a fixed P_9 there are 38 generators (36 transpositions of the nine vertices of W, the reversal, and the color swap), and the base formula has 1,868 variables (459 color variables and 1,409 auxiliaries) and 14,672 clauses.

3.4. **Path clauses generated during the search.** The solver is CaDiCaL [2] with an external propagator attached through the IPASIR-UP interface [10, 11]. The propagator follows the assignment of the color variables. Whenever an edge receives a color c, it searches for paths on m vertices through that edge, in the graph of edges not yet excluded from color c, all of whose edges except at most one are already known to have color c. If all edges are known, it reports the path clause as a conflict; if exactly one is unknown, it sets that edge to "not c" with the path clause as the reason. When the solver reaches a complete assignment, the propagator searches it exhaustively

for a monochromatic P_m and, if it finds one, rejects the assignment with that path clause. Every clause handed to the solver is a path clause and is written to a log.

The refuted formula is therefore the base formula plus the logged path clauses (plus a cube's unit clauses), a subset of the formula with *all* path clauses. A refutation of the subset refutes the full formula; nothing about the completeness of the propagator is needed. Conversely, a satisfying assignment is accepted only after the exhaustive search, and every coloring the method reports is rechecked by a separate program.

4. The ladder

The rung "P_m on K_n, fixing P_k" is the formula of Section 3 with the fixed path on $0, \dots, k-1$, together with all path clauses for P_m. If it is unsatisfiable and every 3-coloring of K_n contains a monochromatic P_k, then by Lemmas 3.1 and 3.2 every 3-coloring of K_n contains a monochromatic P_m. Every 3-coloring of a larger complete graph then does too, by restriction.

Table 2 lists the rungs. The first fixes nothing; each later rung fixes the path guaranteed by the rung before it. Each rung up to P_9 on K_{17} is solved as one formula, with a DRAT proof. The rungs re-derive the upper bounds $R_3(P_m) \le n$ for $m = 3, \dots, 9$, the upper halves of the values in Table 1; the top rung, P_{10} on K_{18}, uses the bound $R_3(P_9) \le 17$ of Dybizbański, Dzido and Radziszowski [7] as re-derived by the rung below it.

Rung	fixed path	variables	base clauses	logged path clauses
P_3 on K_5	none	110	828	11
P_4 on K_6	P_3	92	527	31
P_5 on K_9	P_4	308	2,135	185
P_6 on K_{10}	P_5	368	2,502	1,683
P_7 on K_{13}	P_6	828	6,234	10,094
P_8 on K_{14}	P_7	928	6,897	60,734
P_9 on K_{17}	P_8	1,720	13,633	520,545
P_{10} on K_{18}	P_9	1,868	14,672	(per cube)

Table 2. The ladder. Each rung from P_3 to P_9 is refuted as a whole; the largest takes a few CPU-minutes. The K_{18} rung is split into cubes (Section 5).

The same code also finds colorings where they exist. For example it finds good colorings for P_5 on K_8, P_6 on K_9, P_7 on K_{12} and P_8 on K_{13}, and, with no fixed path, a coloring of K_{16} with no monochromatic P_9; each was confirmed by a separate exhaustive path search.

5. Cube and conquer on K_{18}

5.1. **The cubes.** A cube is a conjunction of literals fixing the colors of some edges. The cubes were produced by the solver itself, with the propagator attached. Walking the edges in edge order, at each edge it assumes each color in turn and runs unit propagation (including the path propagator); colors leading to a conflict are dropped, an edge with a single surviving color is fixed without counting as a split, and the walk stops after 10 edges with at least two surviving colors. This produced 15,316 cubes of 11 to 20 literals, and 18 path clauses learned during generation.

Sixteen of these cubes, each with 11 literals, were much harder than the rest. Each was split once more in the same way, by four further split edges, into 41 children, 656 in all (144 more path clauses were learned during this split). The final list consists of the 15,300 remaining first-level cubes and the 656 children: 15,956 cubes.

5.2. **Why the cubes cover every case.** Cubes whose propagation fails are dropped during generation, so the list is not complete by construction. Its completeness is proved instead by one more certificate: the *cover formula*, which consists of the base formula, the 162 path clauses learned during generation, and the negation of every final cube (one clause per cube), 30,790 clauses in all, is unsatisfiable.

Lemma 5.1. *Let F be a formula, $\mathcal{C}$ a list of cubes, and $\mathcal{P}$ the set of all path clauses. If $F \cup \mathcal{P} \cup \{\neg c : c \in \mathcal{C}\}$ is unsatisfiable and $F \cup \mathcal{P} \cup c$ is unsatisfiable for every $c \in \mathcal{C}$, then $F \cup \mathcal{P}$ is unsatisfiable.*

Proof. A model of $F \cup \mathcal{P}$ satisfies some cube $c \in \mathcal{C}$, since otherwise it falsifies a literal of every cube and so satisfies every $\neg c$. It is then a model of $F \cup \mathcal{P} \cup c$. □

With F the K_{18} base formula, Lemma 5.1 and the 15,957 certificates give the unsatisfiability of the top rung, and the ladder then gives $R_3(P_{10}) \leq 18$. In Lean this is the theorem combining the cover certificate with the per-cube certificates (`unsat_of_cubes`); the per-cube facts enter through `cubes_of_range`, which turns a theorem `K18.all : ∀ i, 0 ≤ i → i < 15956 → CubeOK i` over the list's indices into the statement for every member of the list, given that the list has length 15,956.

6. The Lean formalization

6.1. **The statement.** The definitions use Mathlib's unordered pairs `Sym2`; a coloring of K_n assigns a color to every pair, and the diagonal pairs are never used.

```
abbrev Col (n : ℕ) := Sym2 (Fin n) → Fin 3

def HasMonoPath (n m : ℕ) (X : Col n) : Prop :=
  ∃ (c : Fin 3) (p : Fin m → Fin n), Function.Injective p ∧
    ∀ (i : ℕ) (h : i + 1 < m), X s(p ⟨i, by omega⟩, p ⟨i + 1, h⟩) = c

noncomputable def ramsey3Path (m : ℕ) : ℕ :=
  sInf {n | ∀ X : Col n, HasMonoPath n m X}

theorem K18_has_mono_P10 (X : Sym2 (Fin 18) → Fin 3) :
    ∃ (c : Fin 3) (p : Fin 10 → Fin 18), Function.Injective p ∧
      ∀ (i : ℕ) (h : i + 1 < 10), X s(p ⟨i, by omega⟩, p ⟨i + 1, h⟩) = c

theorem K17_no_mono_P10 : ¬ HasMonoPath 17 10 k17Col

theorem R3_P10_eq_18 : ramsey3Path 10 = 18
```

A path is an injective map p from $\{0, \ldots, m-1\}$ to the vertices whose consecutive pairs all have color c, which is the usual (not necessarily induced) path on m vertices. `ramsey3Path m` is the least n for which every coloring of K_n has a monochromatic P_m; the proof of `R3_P10_eq_18` shows that 18 belongs to that set and that no smaller n does, since a good coloring of K_{17} restricts to one of any smaller complete graph.

6.2. **What is proved.** Lean defines a closed-form encoder `encode n k`, which produces the formula of Section 3, and proves for all n and k that it is sound for the colorings: a coloring with the fixed path that satisfies every generator inequality, with the auxiliaries set to their meaning, satisfies every clause (`encode_sat`); a good coloring satisfies every path clause (`paths_ok`). It proves Lemma 3.1 (`relabel`), that every generator preserves the fixed path and goodness (`gens_fix`, `act_noMono`), Lemma 3.2 (`orbit_min`), the rung argument (`rung`), Lemma 5.1 (`unsat_of_cubes`), the chain of rungs, and the lower bound (Section 2). The facts about formulas are stated with the DIMACS semantics: a literal is a nonzero integer, an assignment is a predicate on variables, and

`UnsatWithPaths n m F`

says that F together with *every* path clause for P_m in K_n has no model.

The proof of `K18_has_mono_P10` uses nine hypotheses of this kind: one for each of the seven rungs, one for the cover formula `encode 18 9 ++ finalCubes.map negCube`, and one for every cube c of `finalCubes`, `encode 18 9 ++ cubeUnits c` (one universally quantified hypothesis over the list, so 15,964 instances in all). The list `finalCubes` is parsed in Lean from the accompanying cube lists, and the cover formula and the cube hypothesis use this same list. The conditional theorems `K18_P10_of_certs` and `R3_P10_eq_18_of_certs` take the nine facts as hypotheses and depend only on Lean's standard axioms `propext`, `Classical.choice` and `Quot.sound`, as does `K17_no_mono_P10`.

6.3. **How the certificates enter.** Each instance is proved by a Boolean check `checkUnsat n m F paths proof` and its soundness theorem

```
checkUnsat_sound : checkUnsat n m F paths proof = true → UnsatWithPaths n m F,
```

proved in Lean. The check confirms that every literal of F is nonzero; that each entry of `paths` (a color and m distinct vertices below n) yields a path clause, which Lean rebuilds from the path itself; and that Lean's verified LRAT checker (`Std.Tactic.BVDecide.LRAT`, with its soundness theorem) accepts `proof` as a refutation of F together with those path clauses. The bridge between the two semantics is a proved lemma (`toStd_eval`): under the nonzero-literal condition, an assignment satisfying the formula in the DIMACS semantics above gives an assignment satisfying its translation into the checker's clause type, so the checker's soundness theorem yields `UnsatWithPaths n m F`, the same predicate that the rung and cover arguments consume. Only the path clauses that the trimmed proof uses are included. Nothing outside Lean decides what counts as a path clause, and the formula checked is a Lean term, not a file: the base formula used in the cube checks is parsed from the accompanying K_{18} formula and proved equal to `encode 18 9` by a comparison that is itself natively evaluated.

Each certificate is stored as a compact string (an LRAT proof with delta-coded hints and implicit clause identifiers). It is decoded by total functions written in Lean. The decoder is outside the logical trusted base: whatever it produces, acceptance implies unsatisfiability by `checkUnsat_sound`. The check is a closed Boolean term, and the kernel accepts its value through `Lean.ofReduceBool`, which evaluates it with Lean's compiled code. The 15,956 cube facts are checked in 162 modules, each covering a contiguous range of indices; a final module joins the ranges into `K18.all`.

6.4. **Axioms and trusted base.** The axiom report of both main theorems is

```
'R3P10.K18_has_mono_P10' depends on axioms:
  [propext, Classical.choice, Lean.ofReduceBool, Lean.trustCompiler, Quot.sound]
'R3P10.R3_P10_eq_18' depends on axioms:
  [propext, Classical.choice, Lean.ofReduceBool, Lean.trustCompiler, Quot.sound]
```

The two non-standard axioms come from the natively evaluated certificate checks (Lean v4.32.2, pinned in the accompanying project). The trusted base therefore has three parts: Lean's kernel, which checks all the mathematics, including the lower bound and the reduction from colorings to formulas; Lean's compiler and interpreter, whose evaluation of the closed Boolean certificate checks the kernel accepts without reducing them itself (`Lean.ofReduceBool`, `Lean.trustCompiler`); and the compiled modules a build imports, which Lean does not recheck. The evaluated code is the verified LRAT checker, the decoder and the clause and path builders; by `checkUnsat_sound`, only the faithful execution of this code is trusted, not its correctness. Lean does not recheck imported compiled modules, so a reader who wants the whole proof checked on one machine rebuilds the 162 certificate modules (Section 8). The SAT solver, the proof converter, the external checkers and the scripts that produced the certificates are not part of the trusted base of the Lean theorem.

Planted errors are rejected by the build: a negated literal in a proof step, a dropped hint, a path with a repeated vertex, a certificate checked against the wrong rung's formula, and a comparison

of the K_{18} encoder with the K_{17} formula each make the natively evaluated check return false, so the theorem is not accepted; a missing range of cubes makes `K18.all` fail to cover 15,956 indices. Seven mutations of the formalization each make the build fail in the expected declaration: a path clause missing an edge, an encoder missing a fixed-path unit (no longer the certified formula), a generator that moves the fixed path, a fixed path one edge longer than the premise provides, a reversed lex comparison, a comparison that ignores the color permutation, and a recolored edge of the K_{17} coloring. Some of these changes are sound in isolation (a dropped clause, or reversing every comparison); their failure shows that the proofs and the certified formulas are tied to each other, not that every sound variant would be accepted.

7. Checks independent of Lean

The result was also checked by a separate route that does not use Lean.

- *Certificates.* CaDiCaL wrote a DRAT proof for each rung and each cube; drat-trim [31] converted it to LRAT [6], and `cake_lpr` [29], a checker verified in CakeML down to its machine code, accepted every rung, every one of the 15,956 cubes and the cover formula.
- *Inputs.* For each check, an input checker confirmed that the file given to `cake_lpr` is exactly the base formula, plus the solver's clause log, plus the cube's unit clauses (or the negated cubes, for the cover).
- *Path clauses.* A separate checker, sharing no code with the encoder or the solver, confirmed that every logged clause consists of $m-1$ distinct negative literals of one color whose edges form a path on m distinct vertices. It rejects ten kinds of planted corruption.
- *Replays.* The solver was deterministic under the pinned build. Every cube was solved a second time on a different computer from the one that first solved it, with byte-identical proofs and clause logs; re-solving all cubes again to produce the Lean certificates reproduced every proof and log once more.
- *Encoding audit.* An independently written auditor, which does not import the encoder, checks every clause of every rung's base formula against a direct reading of the problem (with the auxiliaries at their meaning), checks that the clause multiset is exactly the one the stated encoding generates, with nothing missing or extra, checks that every generator preserves the fixed path, checks the reductions exhaustively on K_4 and K_5 (where the formula accepts exactly the colorings it should and no class of good colorings is lost), and rejects 30 planted corruptions of the formulas, logs and reductions.
- *Lower bound.* An exhaustive search confirms that the K_{17} coloring has no monochromatic P_{10} (and that it has a monochromatic P_9).

Table 3 compares the two routes. In the route of this section the faithfulness of the encoding rests on the auditor and on short written lemmas; in the Lean route it is proved.

8. Computation and replay

8.1. **Cost.** The search took about 217 CPU-hours of solving and 51 of proof checking with `cake_lpr`; the hardest single cube took about 2.9 hours to solve. For the Lean certificates, re-solving every cube took about 185 CPU-hours, conversion to LRAT with drat-trim about 45, and the Lean checks about 15: about 250 CPU-hours in all, spread over independent pieces. The DRAT proofs of the cubes total about 58 GB. They are not distributed: the solver is deterministic, and the replay regenerates them.

8.2. **How to check.** The accompanying files contain the Lean project in `lean/`, the certificate checkers in `checks/`, and the formulas and cube lists in `data/`. They pin the Lean toolchain and Mathlib, and the commits of CaDiCaL, drat-trim and `cake_lpr` that the checks build. The Lean project is checked with the standard commands:

Step	Separate route	Lean route
Each rung and cube, and the cover formula, is unsatisfiable	`cake_lpr` (verified)	verified LRAT checker, native evaluation
The checked formula is the intended one	input checker	the formula is the Lean term
Every logged clause is a path clause	path-clause checker	rebuilt from paths and proved
The base clauses hold for admissible colorings	auditor, clause by clause	proved for all n, k
Relabeling to fix the path (Lemma 3.1)	written; exhaustive on K_4, K_5	proved
Generators preserve the fixed path	auditor, every generator	proved
Lex-leader soundness (Lemma 3.2)	written; exhaustive on K_4, K_5	proved
Logged clauses suffice (monotonicity)	written	proved
Cover argument (Lemma 5.1) and assembly	written	proved
Lower bound	exhaustive search	proved (kernel decision)

TABLE 3. What each route checks by machine and what it argues in writing.

```
cd lean
lake exe cache get
lake build
```

This builds the encoding proof, the conditional theorems, the lower bound and the statement, and checks their axioms; it needs no generated certificate data and does not by itself establish the value. The computation is replayed with

```
python3 checks/check.py quick
python3 checks/check.py full
```

The quick mode re-solves the ladder and checks it, the cover and a random sample of cubes both in Lean and with `cake_lpr`, and runs the planted errors; it prints the seed of its sample. The full mode re-solves and checks every cube in Lean (about 250 CPU-hours, in independent pieces that can run in parallel on several machines), then the ladder and the cover, and finally runs `lake build R3P10Final`, which proves `R3_P10_eq_18` and prints its axioms as in Section 6. With `-cross-check` it also checks every cube with the path-clause checker, the input checker and `cake_lpr`.

ACKNOWLEDGMENTS

This work rests on that of Dybizbański, Dzido and Radziszowski, who settled $n = 8$ and $n = 9$, completing the known values; the computation re-derives the upper bounds for $n \leq 9$, including their $R_3(P_9) \leq 17$, and uses that bound, and on Radziszowski's survey, which records the problem's status and the sources of the earlier values. It uses the SAT solver CaDiCaL and its IPASIR-UP interface, drat-trim, `cake_lpr` and CakeML, Lean 4 and the LRAT checker in its standard library, and Mathlib; we thank their authors and contributors.

REFERENCES

[1] H. Böving, S. Bhat, L. Cicolini, A. Keizer, L. Frenot, A. Mohamed, L. Stefanesco, H. Khan, J. Clune, C. Barrett and T. Grosser, *Interactive bitvector reasoning using verified bit-blasting*, Proc. ACM Program. Lang. **9** (OOPSLA2) (2025), 3259–3285. doi:10.1145/3763167.

[2] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks and F. Pollitt, *CaDiCaL 2.0*, in: Computer Aided Verification (CAV 2024), Lecture Notes in Comput. Sci. **14681**, Springer, 2024, 133–152. doi:10.1007/978-3-031-65627-9_7.

[3] F. S. Benevides and J. Skokan, *The 3-colored Ramsey number of even cycles*, J. Combin. Theory Ser. B **99** (2009), 690–708. doi:10.1016/j.jctb.2008.12.002.

[4] M. Codish, M. Frank, A. Itzhakov and A. Miller, *Computing the Ramsey number $R(4,3,3)$ using abstraction and symmetry breaking*, Constraints **21** (2016), 375–393. doi:10.1007/s10601-016-9240-3.

[5] J. M. Crawford, M. L. Ginsberg, E. M. Luks and A. Roy, *Symmetry-breaking predicates for search problems*, in: Proc. Fifth International Conference on Principles of Knowledge Representation and Reasoning (KR'96), Morgan Kaufmann, 1996, 148–159.

[6] L. Cruz-Filipe, M. J. H. Heule, W. A. Hunt, M. Kaufmann and P. Schneider-Kamp, *Efficient certified RAT verification*, in: Automated Deduction – CADE 26, Lecture Notes in Artificial Intelligence **10395**, Springer, 2017, 220–236. doi:10.1007/978-3-319-63046-5_14.

[7] J. Dybizbański, T. Dzido and S. Radziszowski, *On some three-color Ramsey numbers for paths*, Discrete Appl. Math. **204** (2016), 133–141. doi:10.1016/j.dam.2015.11.002; preprint arXiv:1207.5851 (2012).

[8] T. Dzido and R. Fidytek, *On some three color Ramsey numbers for paths and cycles*, Discrete Math. **309** (2009), 4955–4958. doi:10.1016/j.disc.2008.04.053.

[9] T. Dzido, *Multicolor Ramsey numbers for paths and cycles*, Discuss. Math. Graph Theory **25** (2005), 57–65. doi:10.7151/dmgt.1260.

[10] K. Fazekas, A. Niemetz, M. Preiner, M. Kirchweger, S. Szeider and A. Biere, *IPASIR-UP: User propagators for CDCL*, in: 26th International Conference on Theory and Applications of Satisfiability Testing (SAT 2023), LIPIcs **271**, 2023, 8:1–8:13. doi:10.4230/LIPIcs.SAT.2023.8.

[11] K. Fazekas, A. Niemetz, M. Preiner, M. Kirchweger, S. Szeider and A. Biere, *Satisfiability modulo user propagators*, J. Artificial Intelligence Res. **81** (2024), 989–1017. doi:10.1613/jair.1.16163.

[12] A. Figaj and T. Łuczak, *The Ramsey number for a triple of long even cycles*, J. Combin. Theory Ser. B **97** (2007), 584–596. doi:10.1016/j.jctb.2006.09.001.

[13] R. J. Faudree and R. H. Schelp, *Path Ramsey numbers in multicolorings*, J. Combin. Theory Ser. B **19** (1975), 150–160. doi:10.1016/0095-8956(75)90080-5.

[14] N. F. Galindez, *An upper bound for the three-colour Ramsey number of the ten-cycle: $R(C_{10}, C_{10}, C_{10}) \leq 26$*, Zenodo, 2026. doi:10.5281/zenodo.22000000.

[15] A. Gyárfás, M. Ruszinkó, G. N. Sárközy and E. Szemerédi, *Three-color Ramsey numbers for paths*, Combinatorica **27** (2007), 35–69. doi:10.1007/s00493-007-0043-4; corrigendum, Combinatorica **28** (2008), 499–502. doi:10.1007/s00493-008-2395-9.

[16] M. J. H. Heule, O. Kullmann and V. W. Marek, *Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer*, in: Theory and Applications of Satisfiability Testing – SAT 2016, Lecture Notes in Comput. Sci. **9710**, Springer, 2016, 228–245. doi:10.1007/978-3-319-40970-2_15.

[17] M. J. H. Heule, O. Kullmann, S. Wieringa and A. Biere, *Cube and conquer: Guiding CDCL SAT solvers by lookaheads*, in: Hardware and Software: Verification and Testing (HVC 2011), Lecture Notes in Comput. Sci. **7261**, Springer, 2012, 50–65. doi:10.1007/978-3-642-34188-5_8.

[18] M. J. H. Heule and M. Scheucher, *Happy ending: An empty hexagon in every set of 30 points*, in: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2024), Lecture Notes in Comput. Sci. **14570**, Springer, 2024, 61–80. doi:10.1007/978-3-031-57246-3_5.

[19] R. W. Irving, *Generalised Ramsey numbers for small graphs*, Discrete Math. **9** (1974), 251–264. doi:10.1016/0012-365X(74)90008-9.

[20] M. Kirchweger and S. Szeider, *SAT modulo symmetries for graph generation and enumeration*, ACM Trans. Comput. Log. **25**(3) (2024), Article 18. doi:10.1145/3670405.

[21] S. Letzter, *An improvement on Łuczak's connected matchings method*, Bull. Lond. Math. Soc. **54** (2022), 609–623. doi:10.1112/blms.12587.

[22] L. Maherani, G. R. Omidi, G. Raeisi and M. Shahsiah, *On three-color Ramsey number of paths*, Graphs Combin. **31** (2015), 2299–2308. doi:10.1007/s00373-014-1507-0.

[23] The mathlib Community, *The Lean mathematical library*, in: Proc. 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), ACM, 2020, 367–381. doi:10.1145/3372885.3373824.

[24] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, in: Automated Deduction – CADE 28, Lecture Notes in Artificial Intelligence **12699**, Springer, 2021, 625–635. doi:10.1007/978-3-030-79876-5_37.

[25] G. R. Omidi and G. Raeisi, *On multicolor Ramsey number of paths versus cycles*, Electron. J. Combin. **18** (2011), #P24. doi:10.37236/511.

[26] S. P. Radziszowski, *Small Ramsey numbers*, Electron. J. Combin., Dynamic Survey DS1, revision #18, April 24, 2026. doi:10.37236/21.

[27] B. Subercaseaux, W. Nawrocki, J. Gallicchio, C. Codel, M. Carneiro and M. J. H. Heule, *Formal verification of the empty hexagon number*, in: 15th International Conference on Interactive Theorem Proving (ITP 2024), LIPIcs **309**, 2024, 35:1–35:19. doi:10.4230/LIPIcs.ITP.2024.35.

[28] S. Szeider, *Streaming LRAT certificates into Lean theorems*, preprint, 2026. arXiv:2607.00815.
[29] Y. K. Tan, M. J. H. Heule and M. O. Myreen, *cake_lpr: Verified propagation redundancy checking in CakeML*, in: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2021), Lecture Notes in Comput. Sci. **12652**, Springer, 2021, 223–241. doi:10.1007/978-3-030-72013-1_12.
[30] W. J. Wesley, *New bounds for some small multicolor Ramsey numbers*, preprint, 2025. arXiv:2509.03784.
[31] N. Wetzler, M. J. H. Heule and W. A. Hunt, *DRAT-trim: Efficient checking and trimming using expressive clausal proofs*, in: Theory and Applications of Satisfiability Testing – SAT 2014, Lecture Notes in Comput. Sci. **8561**, Springer, 2014, 422–429. doi:10.1007/978-3-319-09284-3_31.
[32] Yang Yuansheng, *On the third Ramsey numbers of graphs with six edges*, J. Combin. Math. Combin. Comput. **17** (1995), 199–208.
[33] Yang Yuansheng and P. Rowlinson, *On the third Ramsey numbers of graphs with five edges*, J. Combin. Math. Combin. Comput. **11** (1992), 213–222.
[34] Leo-Y-Zhang, *CycleRamsey*, software repository on GitHub, 2026. https://github.com/Leo-Y-Zhang/CycleRamsey.